The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...
2.1 Video Text Guide (Series ...
2.2 One Line of Python Trigge...
2.3 How a 1953 Word Game E...
2.4 The Ladder That Explains ...
2.5 Clawdbot (Moltbot) Has 1...
2.6 Video as Code: My AI Ani...
2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...
3.1 The Full Walkthrough (23 ...
3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...

2.6 Video as Code: My AI Animation Stack

13:12

What You'll Get From This

You will see how one person replaces a week of animation work with under an hour using folders, specs, and a component library. The tools matter less than the process. The process is software engineering applied to creative work.

The written version below covers the same ground as the video. The video above has music and visuals that make the layering easier to follow. The written version is the one you hand to Claude or come back to for reference.

The Lesson

What used to take a week

The animations you see across this channel would have taken a full week, maybe a month, to make by hand. Now they take

you try it, and the output is mediocre, uncanny, or requires so much cleanup that you wonder why you bothered. That is exactly why I want to show you this. The point is not polished results. It is the process. And what you will find is not magic. It is infrastructure. Documentation, specifications, component libraries. Software engineering applied to a creative problem.

The four tools you need

If you wanted to do this yourself, here is what I use:

1. **Claude Code** (or any coding agent like Codex, Aider, etc.)
2. **A code editor** (VS Code, Cursor, whatever you are comfortable with)
3. **Remotion** — a library that turns React components into video, frame by frame
4. **A video editor** — I use CapCut because it is simple and fast

I am not going to walk through installation. There are plenty of tutorials for each of these, and that time is better spent on the actual process. Just know that if you have these tools set up and running, everything I am about to describe is available to you. And the tools themselves are not the point. It is how they work together.

The real secret: writing the spec

Here is the part that surprised me when I started doing this. The hard work is not the AI. It is not the code. It is the spec.

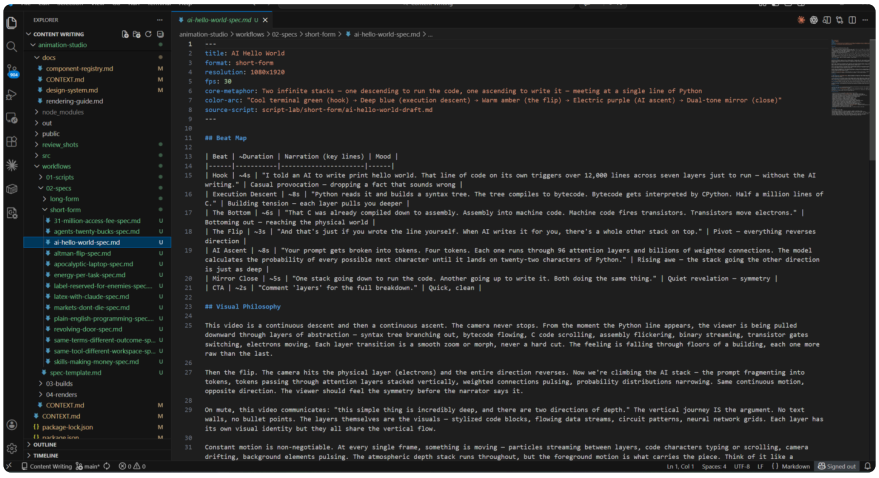
The spec is the markdown file that describes what the video should be. And not vaguely. Not "make it look cool." Precisely. What is the structure? What happens in each scene? What is the timing? When do visual elements appear? What should be emphasized? What should stay in the background?

If you have ever worked with a designer or an animator, you know this as a brief. There is a quote from David Ogilvy: "Give me the freedom of a tight brief." It sounds like a contradiction,

they are going. They can focus on solving the problem instead of trying to figure out what the problem even is.

The spec is where your creative thinking lives. What is the thesis of the video? What is the arc? What are the moments that need to land? What should feel fast and what should feel slow?

A loose spec means Claude makes more interpretive choices. It guesses more. A tight spec means you are directing at every beat. You can work either way depending on where you want your creative energy to go, but the tighter the spec, the closer the output is to what you had in mind.



What a spec file actually looks like

It is a markdown file. Headings for each scene. Descriptions of what happens. Notes on timing and emphasis. Nothing fancy. But it is the document that makes everything else possible.

And this is not just for creative work. This same approach works for code, for client deliverables, for anything you are building with an AI agent. The spec is always the leverage point. You will start to notice how powerful this process is across everything you do.

How Claude Code reads everything

Claude Code reads the spec. But it also reads your documentation, because the spec references other files:

A style guide — defines your visual language. Minimal text rules, color palettes, animation timing. Decisions that stay the same across every video.

A component registry — a list of reusable pieces you have already built. Text animations, backgrounds, transitions, data visualizations. Building blocks that work together.

Then Claude builds each scene one by one. Each scene is a React component. Remotion takes those React components and renders them frame by frame into video.

This is the component library concept from **Section 2.5**. You are not building from scratch every time. You are assembling existing pieces according to a spec. Claude knows the components. It knows the style rules. It composes them into scenes that match what you described.

Why Remotion changes the game

Remotion was created by Jonny Burger. The core insight: a video is just a function of images over time. If you change the content every frame, you get animation. React is already good at describing what should appear based on some state. Normally that state is user input. In Remotion, the state is the current frame number.

You tell React what frame 47 should look like. It renders it. Do that for every frame. You now have a video.

What this means in practice: anything that works in a web browser can be part of your animation. CSS, SVG, canvas, the entire ecosystem of JavaScript libraries. You are not learning a new tool from scratch. If you know any front-end development (or if you have Claude Code writing the front-end for you), you are using tools that have existed for years, applied to a new purpose.

---

2. Point it at the documentation (style guide, component registry)
3. Tell it which scene to build
4. It writes the code
5. Preview it in Remotion Studio
6. If something is not right, describe in natural language what needs to change
7. Claude adjusts. Iterate until the scene matches what you had in mind.

The component library matters because you are not building every animation from scratch. You have building blocks: a text component that handles entrance animations, a scene container that handles safe margins and backgrounds, effects like glitch text or counting numbers. Claude assembles these pieces according to the spec. It is composing, not inventing.

This is separation of concerns. The same principle that makes any complex software manageable. The spec is separate from the implementation. The components are reusable. The style guide encodes decisions so you do not make them every time. Break complex things into smaller pieces that can be understood and changed independently.

If you went through **Section 1.2**, this should sound familiar. The folder architecture does the same thing: identity file, workspace context, skills. Each one handles a different concern. This animation pipeline is the folder architecture applied to a specific production workflow.

Exporting to CapCut for the final edit

Once the scenes are built, I export from Remotion (or just use OBS to record the screen, which avoids render and codec headaches). Then I bring it into CapCut.

honestly faster by hand than describing it in a spec and waiting for code to regenerate.

The animation is not the finished video. It is a substantial starting point. I still spend real time in this phase. I cut sections that do not work when I hear them out loud. I re-record lines that do not land. I add photos, screenshots, or footage to support the narration. The animations give me a foundation. The finished video requires refinement.

So when I say "under an hour," I mean the animation, the part that would have taken a week in After Effects if I was doing it frame by frame. The finished video still takes time. But I am spending that time on the parts I actually want to control: pacing, moments that need a real image, the edit.

The bigger picture: GarageBand for video

What is happening here is part of a much larger pattern.

When GarageBand came out in 2004, Steve Jobs wanted to democratize music making. Suddenly every teenager with a Mac could produce music that would have required a $100,000 studio five years earlier. Steve Lacy made tracks for Kendrick Lamar on a cracked iPhone with GarageBand and a $20 iRig. That phone is now in the Smithsonian.

Canva did something similar for visual design. Melanie Perkins was rejected by over a hundred investors who did not think design tools for non-designers was a real market. Now 200 million people use it monthly.

The pattern is always the same. A tool emerges that gives people access to execution they could not do before. Not replacing professionals. Letting people who have ideas actually realize them.

There is a messy period early on. Designers called it the "ransom note effect" when desktop publishing first appeared. Everyone used 15 different fonts on one page. But over time, the floor rises. More people creating means more good work in

We are in that moment for AI-assisted animation and video. The tools exist. The quality can be good. People who could not afford a motion graphics artist, or could not justify a week of their own time, can now make something real.

Why constraints make it better

There is actual research on this. An inverted U-shaped relationship between constraints and creative output. Too few constraints and you get paralysis (the blank canvas problem). Too many and you get stifled. But in the middle, with the right boundaries, creativity increases.

Dr. Seuss wrote Green Eggs and Ham because his editor bet him $50 he could not write a book using only 50 words. He made flowcharts on his walls to track his vocabulary. The book sold 200 million copies. His best-selling work ever.

Spielberg could not get the mechanical shark to work in Jaws. So he showed floating barrels and point-of-view shots instead. The shark does not fully appear until 80 minutes into the film. The constraint made the movie better.

The spec, the style guide, the component registry: these are not limitations I work around. They are boundaries that make the output better. Claude is the one constrained. It knows the rules. It focuses on execution instead of guessing what I want.

This connects directly to **Section 1.3**. Constraints were part 4 of the prompting framework. Every constraint you give is a mistake Claude will not make. In this animation pipeline, the style guide and component registry are constraints encoded as files. Same principle, applied at a larger scale.

What actually changed

AI did not make the creative work disappear. It moved it.

The spec is where my creative thinking lives. The documentation encodes my preferences and lessons from past mistakes. Claude executes within those boundaries, assembling

One person doing the work that used to require 4 to 6 specialists and 8 to 12 weeks. That is the real claim. Not that AI replaced the creativity. That the system let one person handle the execution.

If you have ideas that have been stuck in your head because you could not figure out how to make them visible, this process can be learned. The tools are freely available. Most of the system is just software engineering applied to creative work: constraints, separation of concerns, reusable components, clear specifications.

Resources

- **Remotion**
- **CapCut**

Go Deeper

- Want to learn the full animation pipeline hands-on? The **Building Animations** course (Level 2) walks you through the Main Tutorial, Illustrator-to-web workflow, and the full course companion with resources. [**Animations Course**]
- The folder architecture from **Section 1.2** is the same system applied broadly. This animation pipeline is one specific implementation of it.
- The prompting framework from **Section 1.3** is running throughout this process. The spec is identity + task + context + constraints. The component registry is encoded output format.
- **Section 2.5** showed the **60/30/10 breakdown**. This animation pipeline follows the same pattern: most of the system is traditional code (components, Remotion config), the routing is in the spec and documentation, and the AI calls are a small fraction of the total work.

and the system around it is where the value lives now.

⏭️ Next Up: 2.7 From Nazi Psychology to AI Auditing

We have covered the technical stack (2.2), the memory model (2.3), where value lives (2.4, 2.5), and how systems replace manual work (2.6). The last video in the series goes somewhere different: the history of psychological measurement instruments, how they ended up running on AI models, and what that means for auditing the systems we are building.

Matthew Creamer
24d • Lesson Check-Ins

Foundations 2.6 Check-In

You just saw how video production turns into code. Vote below, then tell us in the comments: what is one repetitive

Poll 115 members have voted

35 49 New comment 14h ago